

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: INTERNET PROGRAMMING

COURSE CODE: CSA4305

COURSE OUTCOME COVERED

CO5: Construct interoperable web services by leveraging JAX-RPC, WSDL, SOAP, and XML Schema for seamless data exchange across distributed systems.(L4,L5)

Assessment Tool : Alternative Solution Design

Weightage: 20%

QUESTIONS:

Total Marks: 25

1. A company needs to develop a web service for a mobile and web application. Analyze both **REST and SOAP** approaches and design an appropriate solution by selecting one. Justify your choice based on performance, scalability, and security requirements.
2. An online platform expects a large number of users accessing its services simultaneously. Design a **scalable service architecture** showing how client, server, API, and database interact. Propose an alternative design to improve scalability and justify your approach.
3. A banking application requires secure communication between client and server. Analyze available communication protocols and **select the most suitable protocol**. Justify your selection based on security, reliability, and performance.
4. An application needs to integrate with third-party services such as payment gateways or maps. Design an **API integration strategy**, including request handling, authentication, and response processing. Suggest an alternative approach to improve efficiency or reliability.
5. A web service frequently encounters errors during client requests. Design an **error handling and fault tolerance mechanism**. Propose alternative strategies to improve system reliability and ensure smooth user experience.

Code of Conduct:

I P.V.Prakalya (192311418) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Praedye P.v

RUBRICS FOR CALCULATION:

Criteria	Weight age (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	20%	Clearly defines problem, objectives, and all requirements accurately	Understands problem with minor gaps	Partial understanding; misses key requirements	Misinterprets or fails to understand problem
Generation of Solutions	25%	Develops multiple innovative and feasible alternatives with clear differentiation	Generates relevant alternatives with minor limitations	Limited or repetitive alternatives	Fails to generate meaningful alternatives
Evaluation of Alternatives	25%	Critically compares alternatives using appropriate criteria and metrics	Compares alternatives with some justification	Basic or superficial comparison	No proper comparison conducted
Solution Selection	20%	Selects best solution with strong justification and decision rationale	Selects suitable solution with moderate justification	Selection is weakly justified	No clear or justified selection
Feasibility	10%	Applies relevant concepts ensuring high feasibility and practicality	Applies concepts with minor issues	Limited feasibility or incorrect application	Solution is impractical or conceptually incorrect

1. REST vs SOAP for Mobile and Web Application

Problem Understanding

The company needs a web service that can serve both mobile and web applications. The service should provide fast responses, support many users, scale easily, and protect user information. Since mobile applications may operate with limited bandwidth, reducing message size and communication overhead is important.

Alternatives

Alternative 1 – SOAP:

SOAP uses XML-based messages and provides a strict service contract through WSDL. It supports enterprise security and reliability features but generally produces larger messages and requires more processing.

Alternative 2 – REST:

REST uses HTTP methods such as GET, POST, PUT, and DELETE. It commonly uses JSON, which is lightweight and easier for mobile and web clients to process.

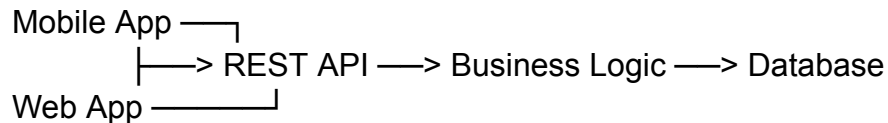
Comparison

Factor	REST	SOAP
Performance	High	Moderate
Message size	Small with JSON	Larger XML messages
Scalability	Excellent	Good
Mobile support	Excellent	Moderate
Security	HTTPS, OAuth, JWT	WS-Security + HTTPS
Complexity	Low	High
Flexibility	High	More rigid

Selected Solution: REST API

REST is the better choice for this application because mobile and web clients require lightweight, fast, and scalable communication. JSON responses reduce bandwidth usage and processing time. REST also works naturally with HTTP caching and load balancing.

A suitable architecture is:



HTTPS, OAuth 2.0/JWT authentication, input validation, rate limiting, and role-based authorization can provide security.

Justification

REST provides better performance and scalability for a large mobile/web application while maintaining strong security through HTTPS and modern authentication mechanisms. SOAP would be preferable if the application required strict enterprise contracts, WS-Security, or complex transactional operations.

Decision: REST API

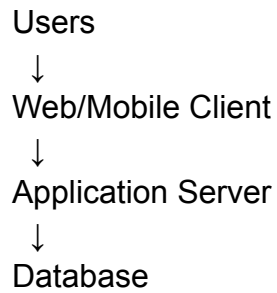
Rubric focus: Multiple alternatives are generated, evaluated using performance/scalability/security, and REST is selected with a practical architecture.

2. Scalable Architecture for a High-Traffic Online Platform

Problem Understanding

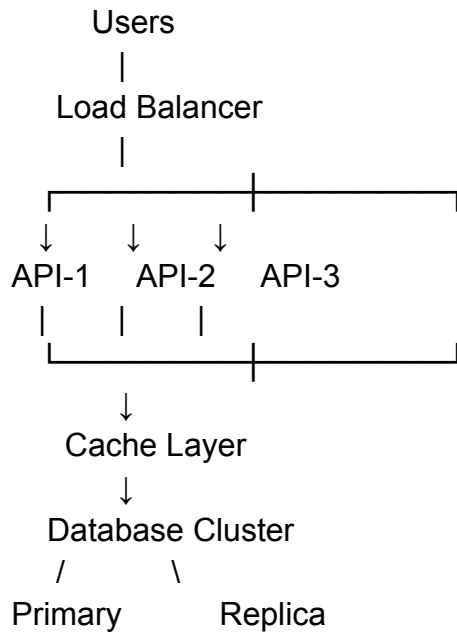
An online platform may receive thousands or millions of simultaneous requests. A single server can become overloaded, resulting in slow responses or service failure. Therefore, the architecture must distribute traffic and database operations efficiently.

Alternative 1 – Basic Three-Tier Architecture



This architecture is simple and easy to implement, but a single application server and database can become bottlenecks.

Alternative 2 – Scalable Distributed Architecture



Comparison

Factor	Basic Architecture	Distributed Architecture
Initial complexity	Low	Higher
Scalability	Limited	Higher
Fault tolerance	Low	Higher
Performance	Moderate	Higher
Maintenance	Easy	More complex
Large user support	Poor	Excellent

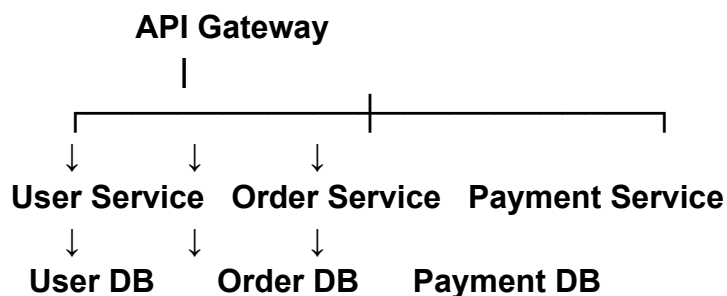
Selected Solution

The distributed architecture with load balancing, caching, and database replication is selected.

The load balancer distributes incoming requests across multiple API servers. If one server fails, requests can be redirected to another server. A cache stores frequently requested information and reduces repeated database queries. Database replication distributes read operations and improves availability.

Alternative Improvement: Microservices

For very large platforms, the API can be divided into independent services:



This allows individual services to scale independently.

Justification

The distributed architecture is more suitable because it removes single points of failure and allows horizontal scaling. Caching reduces database workload, while load balancing distributes traffic.

Decision: Load-balanced API servers + caching + database replication, with microservices as a future extension.

3. Secure Communication for a Banking Application

Problem Understanding

A banking application transfers highly sensitive information such as account details, transaction information, and authentication credentials. Communication must prevent eavesdropping, tampering, replay attacks, and unauthorized access while maintaining reliable performance.

Possible Protocols

Alternative 1 – HTTP:

Simple and fast but transmits information without encryption. It is unsuitable for banking transactions.

Alternative 2 – HTTPS:

HTTPS uses HTTP over TLS and encrypts communication between the client and server.

Alternative 3 – SFTP:

SFTP provides secure file transfer but is designed mainly for transferring files rather than interactive banking API communication.

Comparison

Requirement	HTTP	HTTPS/TLS	SFTP
Encryption	No	Yes	Yes
Web/API support	Excellent	Excellent	Limited
Performance	High	High	Moderate
Authentication	Basic	Strong with TLS + application auth	Strong
Banking suitability	Poor	Excellent	Limited

Selected Protocol: HTTPS with TLS

The most suitable protocol is HTTPS using modern TLS.

Banking App

|

| Encrypted HTTPS/TLS

↓

API Gateway

|

↓

Banking Application

|

↓

Secure Database

Security Mechanisms

- TLS encrypts data during transmission.
- Server certificates help authenticate the server.
- OAuth 2.0/OpenID Connect or another suitable authentication mechanism can manage user authentication.
- Multi-factor authentication provides additional protection.
- Authorization ensures users can access only permitted operations.
- Rate limiting helps reduce automated abuse.
- Transaction logging supports auditing.

Justification

HTTPS provides the best balance between security, reliability, and performance for interactive banking applications. Unlike HTTP, it protects data from network interception and tampering. Compared with SFTP, HTTPS is designed for continuous web/API communication.

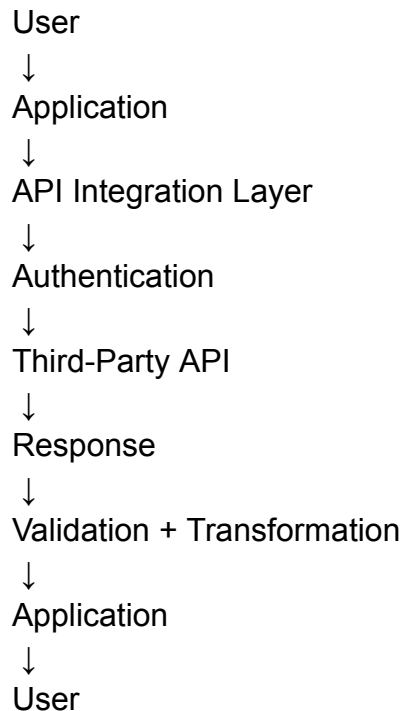
Decision: HTTPS + TLS with strong application-level authentication and authorization.

4. API Integration with Payment Gateway or Maps

Problem Understanding

An application may depend on external services such as payment gateways, map providers, SMS services, or shipping APIs. External services may have different authentication methods, response formats, rate limits, and temporary failures. Therefore, integration should be secure, controlled, and fault tolerant.

Proposed API Integration Strategy



Step 1 – Request Handling

The application creates a request containing only the required information.

For example:

```
{  
  "amount": 1500,  
  "currency": "INR",  
  "order_id": "ORD101"  
}
```

Step 2 – Authentication

The application authenticates with the third-party service using an API key, OAuth token, or another mechanism required by that provider.

Credentials should be stored securely and not hard-coded in source code.

Step 3 – Send Request

The integration layer sends the request using HTTPS.

Application → HTTPS → Payment/Maps API

Step 4 – Response Processing

The response is checked for:

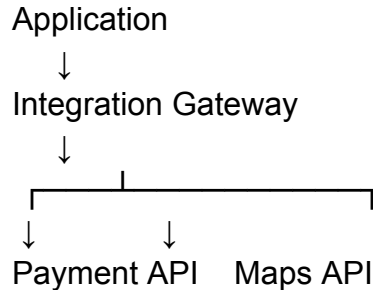
- HTTP status code
- Required fields
- Valid data format
- Error information
- Transaction/reference ID

Example:

```
{  
  "status": "success",  
  "transaction_id": "TXN5001"  
}
```

Alternative Approach: Integration Gateway

Instead of allowing every application component to communicate directly with third-party APIs, an API Gateway/Integration Service can be introduced.



The gateway can handle authentication, logging, retries, rate limiting, request transformation, and monitoring centrally.

Comparison

Approach	Advantage	Limitation
Direct API integration	imple and fast to develop	More difficult to manage many APIs
Integration Gateway	Centralized security, monitoring and retry	Additional infrastructure

Selected Solution

For a small application, direct integration is feasible. For a large enterprise application, the integration gateway is preferred because it provides centralized security, monitoring, rate limiting, and failure handling.

Decision: API Integration Gateway for scalable and reliable third-party integration.

5. Error Handling and Fault Tolerance for Web Services

Problem Understanding

Frequent client requests can fail because of invalid input, network problems, server overload, database failures, or temporary third-party service errors. If failures are not handled properly, users may see confusing error messages or lose transactions.

Alternative 1 – Basic Error Handling

The server validates requests and returns appropriate HTTP status codes.

200 → Successful request
400 → Invalid request
401 → Authentication required
403 → Access denied
404 → Resource not found
500 → Server error
503 → Service temporarily unavailable

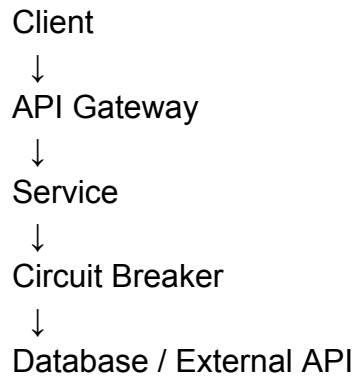
This approach is simple but does not provide strong protection against repeated or temporary failures.

Alternative 2 – Fault-Tolerant Architecture

A stronger approach combines:

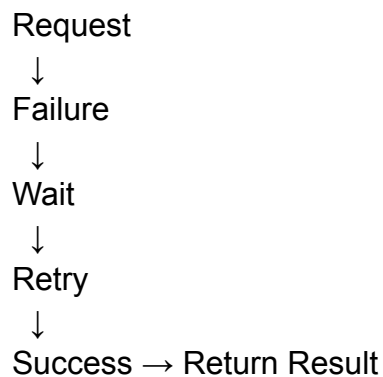
- Input validation
- Centralized exception handling
- Timeouts
- Retry mechanism
- Circuit breaker
- Rate limiting
- Logging
- Monitoring
- Backup services
- Graceful degradation

Architecture:



Retry Mechanism

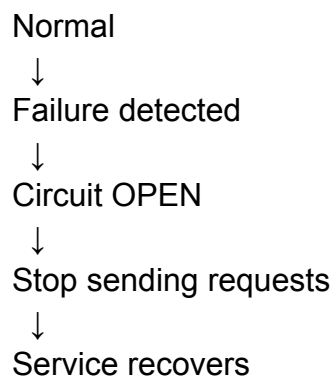
For temporary failures, the service can retry the request with increasing delays.



Retries should be used carefully, especially for financial operations, because repeating a non-idempotent transaction could create duplicate operations.

Circuit Breaker

If a dependent service repeatedly fails:



↓
Circuit CLOSED

This prevents a failing external service from causing cascading failures throughout the application.

User-Friendly Error Response

Instead of displaying a technical error:

ERROR: Database connection exception...

the application can return:

```
{  
  "status": "error",  
  "message": "The service is temporarily unavailable. Please try again."  
}
```

The technical details should remain in secure server logs rather than being exposed to users.

Comparison

Strategy	Reliability	Complexity	Best Use
Basic exception handling	Medium	Low	Small applications
Retry	High for temporary failures	Medium	Network/API failures
Circuit breaker	Very High	Medium	Distributed systems
Redundancy/failover	Very High	Medium	Critical systems

Selected Solution

A combined fault-tolerant mechanism using validation, proper HTTP errors, timeouts, controlled retries, circuit breakers, centralized logging, monitoring, and graceful fallback is the best solution.

This approach prevents individual failures from becoming system-wide failures and gives users clear feedback while allowing the system to recover automatically.

Decision: Layered error handling + retry + circuit breaker + monitoring + graceful degradation.